



CIL-- 합성을 통한 Sparrow 분석기 공격

정원준

2026년 7월 10일

ROPAS@SNU

목차

Sparrow 분석기 공격 목표

CIL-- 언어

실행의미를 Big-Step으로 정의

Sparrow의 실행의미와 대응시키기

메모리를 포함하여 합성하는 방법

앞으로 할 일

Sparrow 분석기 공격 목표

Sparrow 공격 목표

■ completeness 공격:

abstract semantics가 concrete semantics를 너무 크게 포함함
(concrete = abstract인 그림) (concrete $\subset$ abstract인 그림)
(abstract = top인 그림)

Sparrow 공격 목표

■ completeness 공격:

abstract semantics가 concrete semantics를 너무 크게 포함함
(concrete = abstract인 그림) (concrete $\subset$ abstract인 그림)
(abstract = top인 그림)

■ soundness 공격:

concrete semantics가 abstract semantics에 포함되지 않음(버그)
(concrete $\not\subset$ abstract인 그림)

Sparrow의 기본 구조

(C->CIL->CFG 의 예시 그림)

예시는

```
int main() { int x = 1; if(x == 1) x++; return x; }
```

정도로

CIL-- 언어

CIL 언어

CIL이 뭐냐, C의 부분집합 느낌. 분석을 쉽게 할 수 있는 언어
애바하거나 복잡한 구조 단순화, side effect 제거, 정규화

CIL- 언어

completeness를 공격하기 위해서는 튜링 완전한 subset이기만 하면
충분 soundness를 공격하기 위해서는 많은 feature가 있을수록 좋음
(버그 찾는거니까)

CIL- : 의미적으로 별로 필요없는것들 제거 (typedef, struct 등) CIL- :
나중에 추가했으면 좋겠지만 일단 미니멀하게 가기 위해 제거 (포인터 +
malloc/free, 배열, int 외 type)

실행의미를 Big-Step으로 정의

에

CompCert의 Clight Big-Step 의미구조가 있긴한데, 여기서는
CIL-에서의 의미구조를 직접 정의

expression은? 메모리를 보면서 정의 메모리는? stack(frame)과 heap
이 있는데, 아직 heap은 없음

call은? frame을 하나 넣고, arg랑 로컬 변수 할당, return하는거까지
표현 (그림 넣기)

으에

동치성 ast 동치, bigstep tree 동치, 메모리 동치 이게 언제 필요하지?

Sparrow의 실행의미와 대응시키기

에

C->CIL->CFG ->CIL- 구조. C쪽은 더 이상 볼 필요가 없음

가장 큰 문제: sparrow는 concrete semantics를 정확히 정의하지 않음.

C: ISO C 표준이 있는데, formal하진 않음 CIL: 그런것도 없음.

Sparrow: 애도 아예 없음 공격기: CIL에 대한 Big-Step tree랑,
 derivator가 있긴 함

공격기의 concrete 실행의미가 Sparrow의 abstract 실행의미에
 포함되지 않더라도, soundness가 깨졌다! 라고 말할 수 있는가?
 (여기에 그림 넣기)

C 언어의 비결정성 보기

각종 비결정성을 모두 제거해야 공격을 제대로 할 수 있음

C 언어의 비결정성

예	C	CIL	CIL--
h(f(),g()) f()+g() {f(),g()}	계산 순서 비결정	함수 호출은 expression으 로 들어가지 않음	OK
sizeof(int[n++])	n++을 실행하거나/하지 않거나	n++ 문법이 없음	OK
i = i++ + 1;	undefined behavior	i++ 문법이 없음	OK
int x; use(x);	indeterminate value	indeterminate value	합성 시 생성 x
"a" == "a"	true/false 모두 가능	여전히 비결정적	string 없음
char c = -1;	-1/255	여전히 비결정적	char 없음
x >> 1	x < 0 일 때,	여전히 비결정적	비트연산 없음
malloc(n)	메모리 주소가 비결정적	fresh object	fresh object

C 언어의 비결정성

예	C	CIL	CIL--
h(f(),g()) f()+g() {f(),g()}	계산 순서 비결정	함수 호출은 expression으 로 들어가지 않음	OK
sizeof(int[n++])	n++을 실행하거나/하지 않거나	n++ 문법이 없음	OK
i = i++ + 1;	undefined behavior	i++ 문법이 없음	OK
int x; use(x);	indeterminate value	indeterminate value	합성 시 생성 x
"a" == "a"	true/false 모두 가능	여전히 비결정적	string 없음
char c = -1;	-1/255	여전히 비결정적	char 없음
x >> 1	x < 0 일 때,	여전히 비결정적	비트연산 없음
malloc(n)	메모리 주소가 비결정적	fresh object	fresh object

이외에도 malloc 실패 여부, struct의 byte 패딩, 정수 overflow 등 여러
가지 고려해야 할 비결정성이 많음

메모리를 포함하여 합성하는 방법

에

그러게 어떻게 하지... 증명나무 조각들을 레고처럼 끼워서
이어붙이려면 sub-prooftree의 모든 구조가 다 제대로 있어야 함. 즉,
전/후 메모리가 잘 정의되어 있어야 함 같은 expression "x"여도, 메모리
하나당 한 개의 proof tree가 생기고 이들은 서로 구분됨.

앞으로 할 일

에

syntax, semantics(big-step), interpreter 등등 기본적인거 짜 났으니
이제 본격적으로 합성 시작.

걱정거리: simple.c만 해도 크기가 매우매우 큼. (그림 삽입) 재귀적으로
합성할 것/아닌 것을 구분해서 쉬운 step은 바로바로 진행되게끔 하는
스킬이 필요할 듯

감사합니다